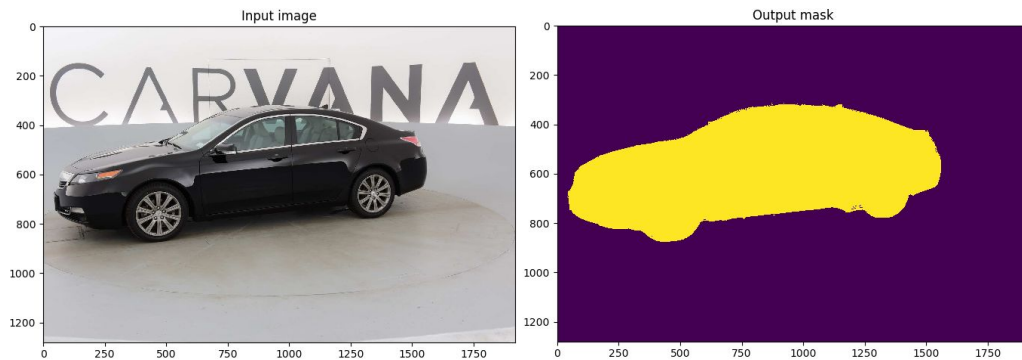


Ускорение инференса UNet модели

Бельский Антон
Курилов Олег
Комогорцев Андрей

Немного про задачу

Dataset: Milesial/PyTorch U-Net: Semantic segmentation auto



Train size ~ 5000

Image resolutions (1918, 1280)

Target metric: Dice

План проделанной работы

- Ускорение инференса с помощью компиляторов (Андрей)
- Квантизация (Антон)
- Спарсификация и прунинг (Олег)

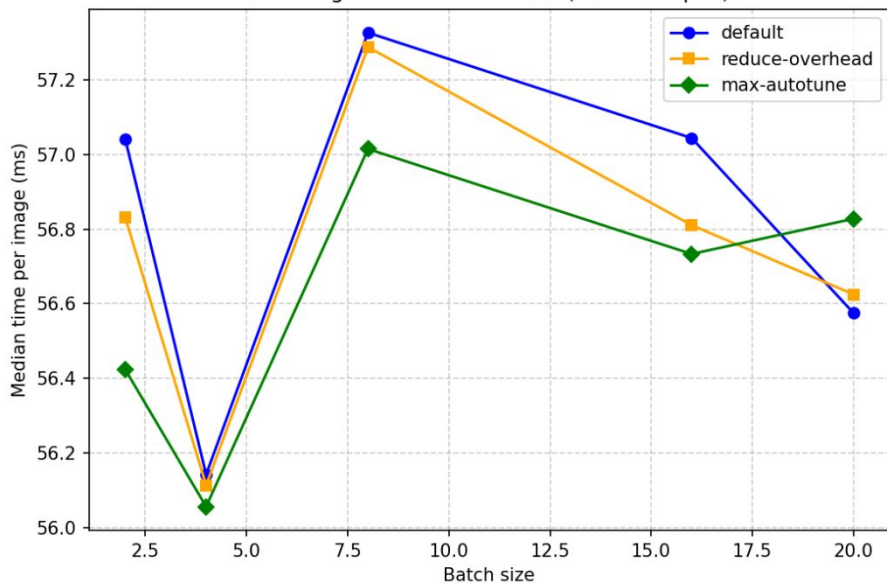
Использованные методы компиляции

- `torch.compile` (фиксированный backend 'inductor')
- `TensorRT` (AOT компилятор, через python API 'torch_tensorrt')
- `ONNX Runtime, ORT` (JIT оптимизация IR, AOT компиляция через EP)
- `TVM` (возникли сложности сборки и наличие багов в интерфейсе создания IR)

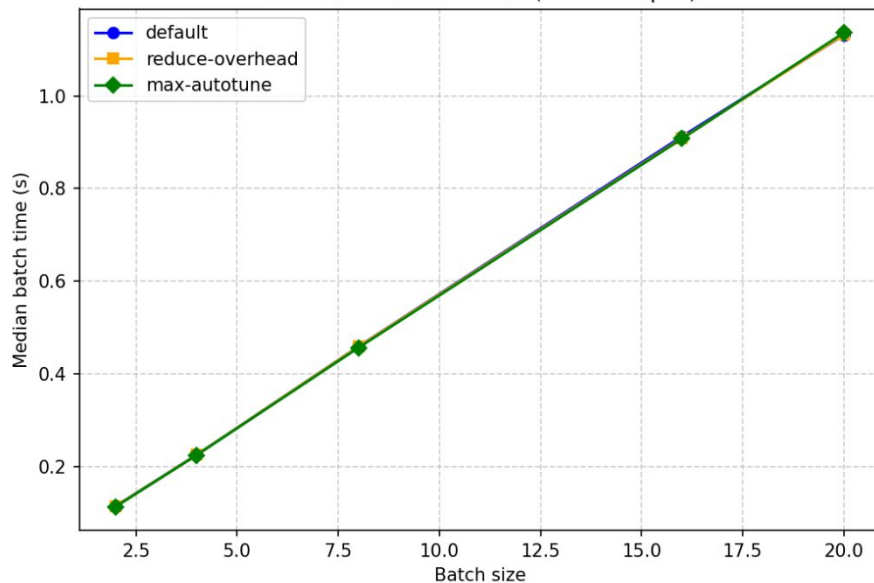
Set-up: Tesla T4, CUDA 13.0

torch.compile(backend='inductor')

Per-image time vs batch size (torch.compile)



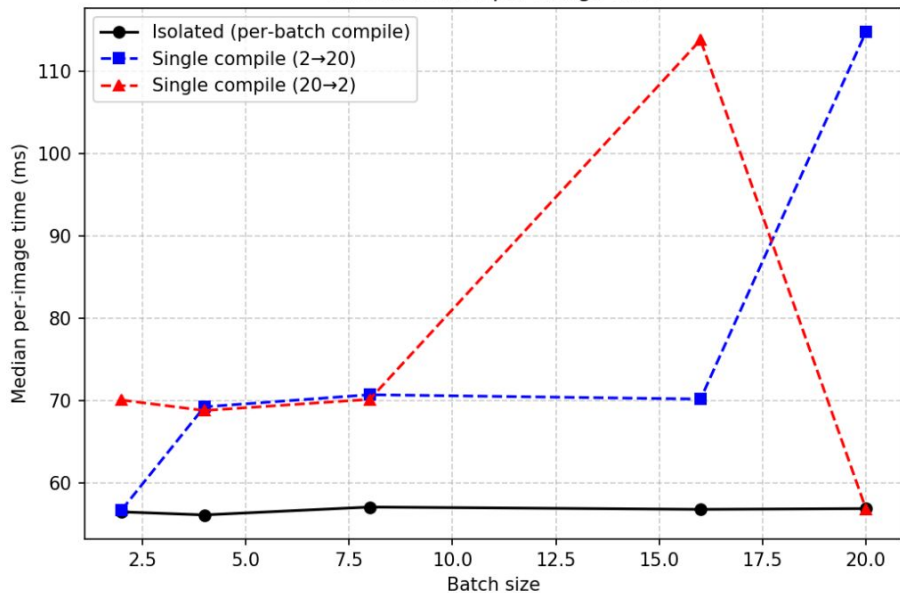
Batch time vs batch size (torch.compile)



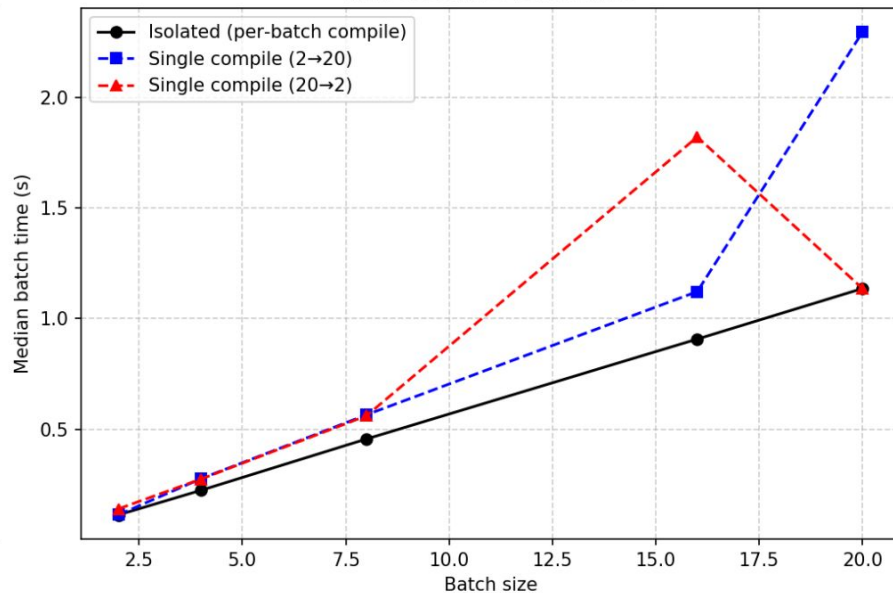
Практически нет разницы между разными 'mode' для компиляции

`torch.compile(backend='inductor', mode='max-autotune')`

Max-autotune: per-image time

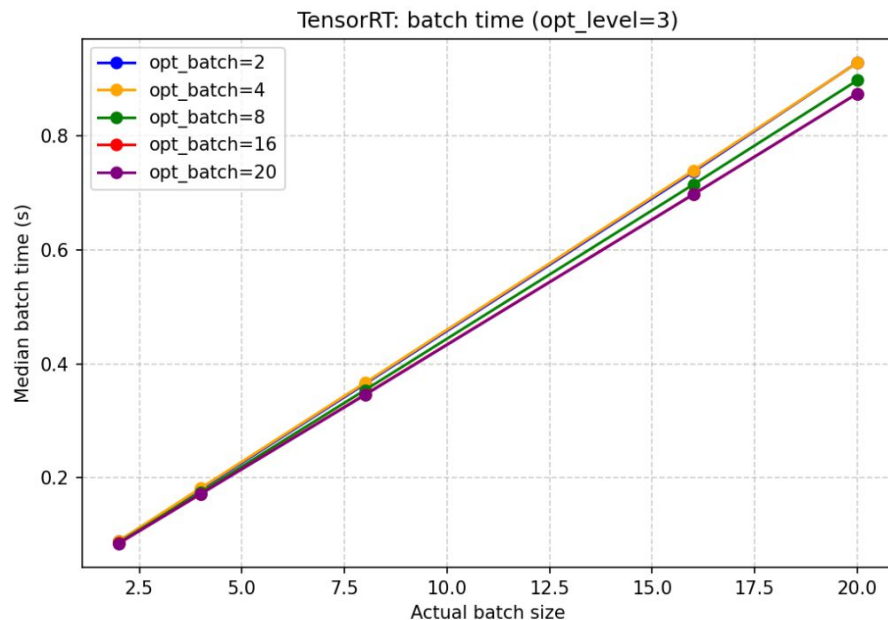
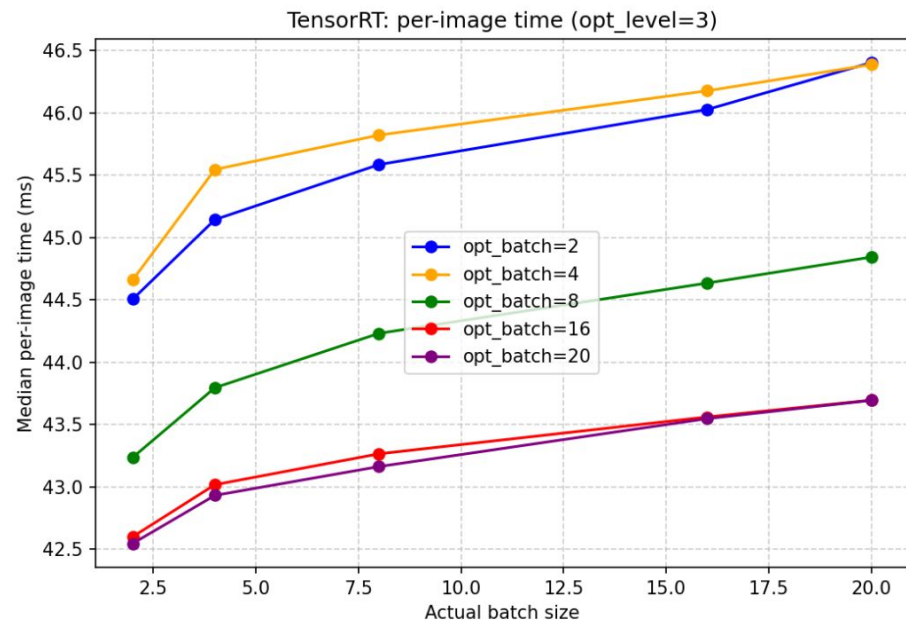


Max-autotune: batch time



`torch.compile` не устойчив в динамическим изменениям `batch_size`

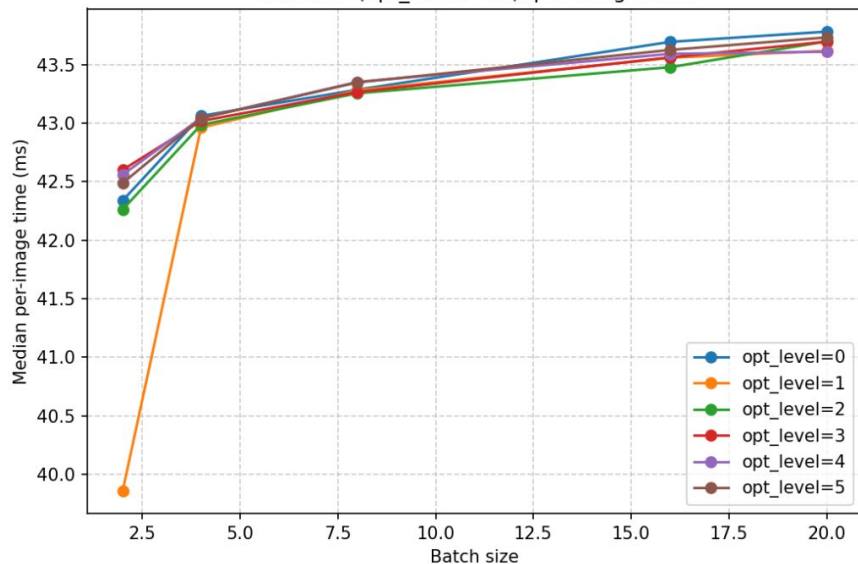
Tensor RT (opt_lvl=3), AOT компиляция



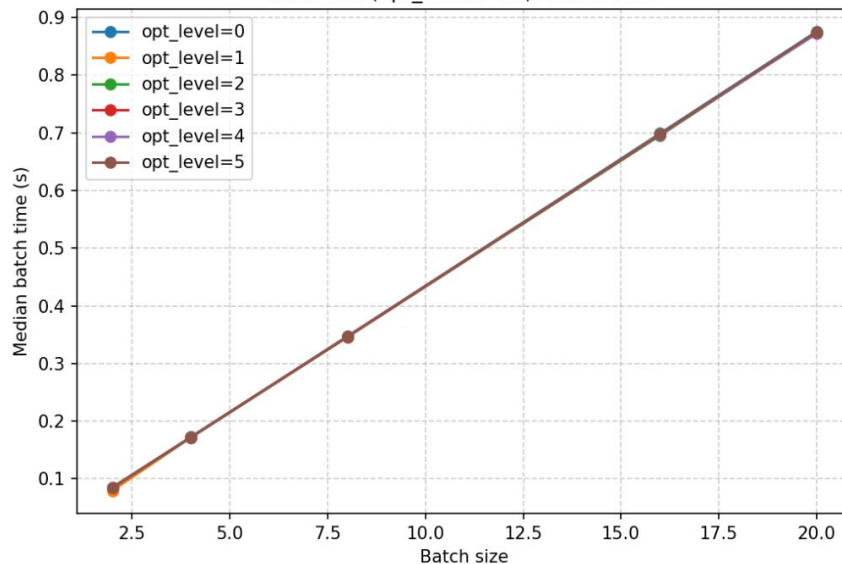
Лучшая конфигурация получается при opt_bsize 16 и 20 (прирост 5%)

Tensor RT (opt_batch=16), AOT компиляция

TensorRT (opt_batch=16): per-image time

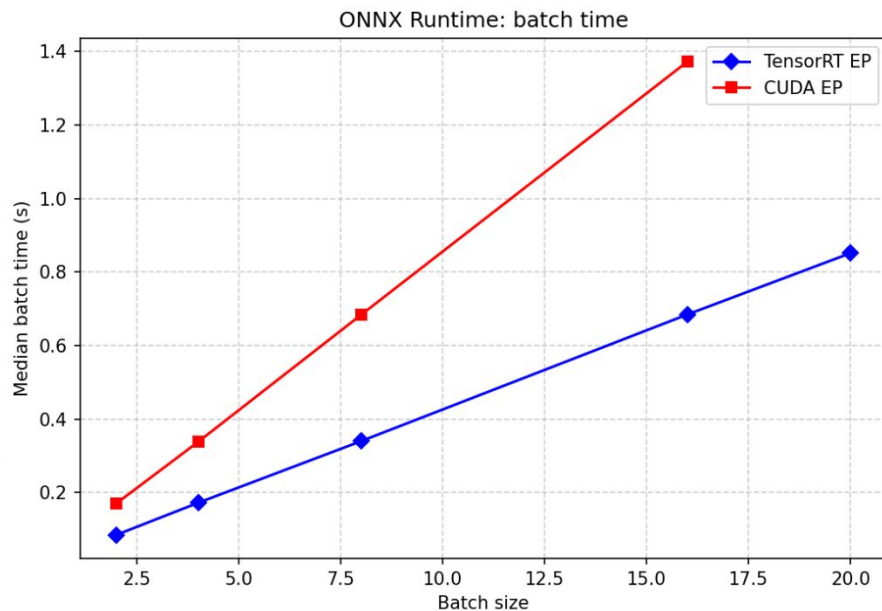
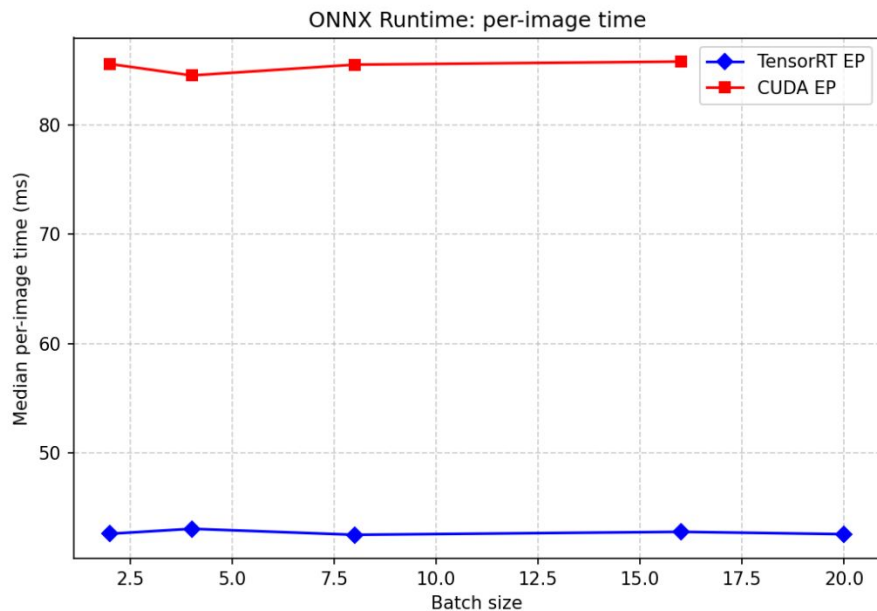


TensorRT (opt_batch=16): batch time



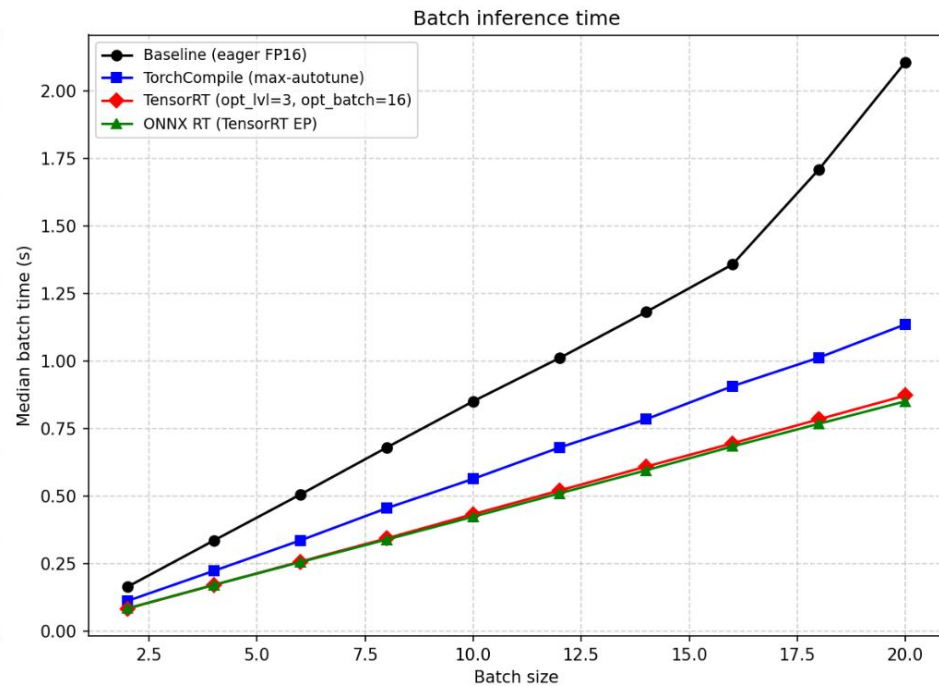
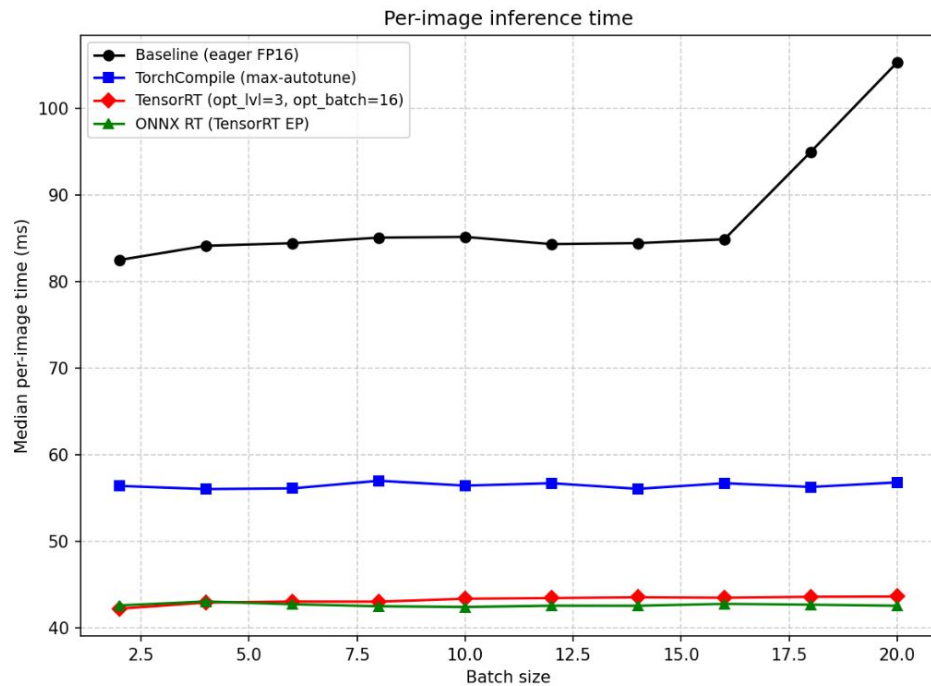
Разницы практически нет (мало SM блоков, у T4 их порядка 40)

ONNX Runtime, ORT (JIT фронтенд, AOT бэкенд)



CUDA Execution Provider, показывает себя хуже чем Tensor RT

Итоговое сравнение инференса



Лучшие показатели у ONNX RT и Tensor RT (движок копирования от NVIDIA)

Полученное ускорение и оценка качества Dice

Спарсификация и прунинг

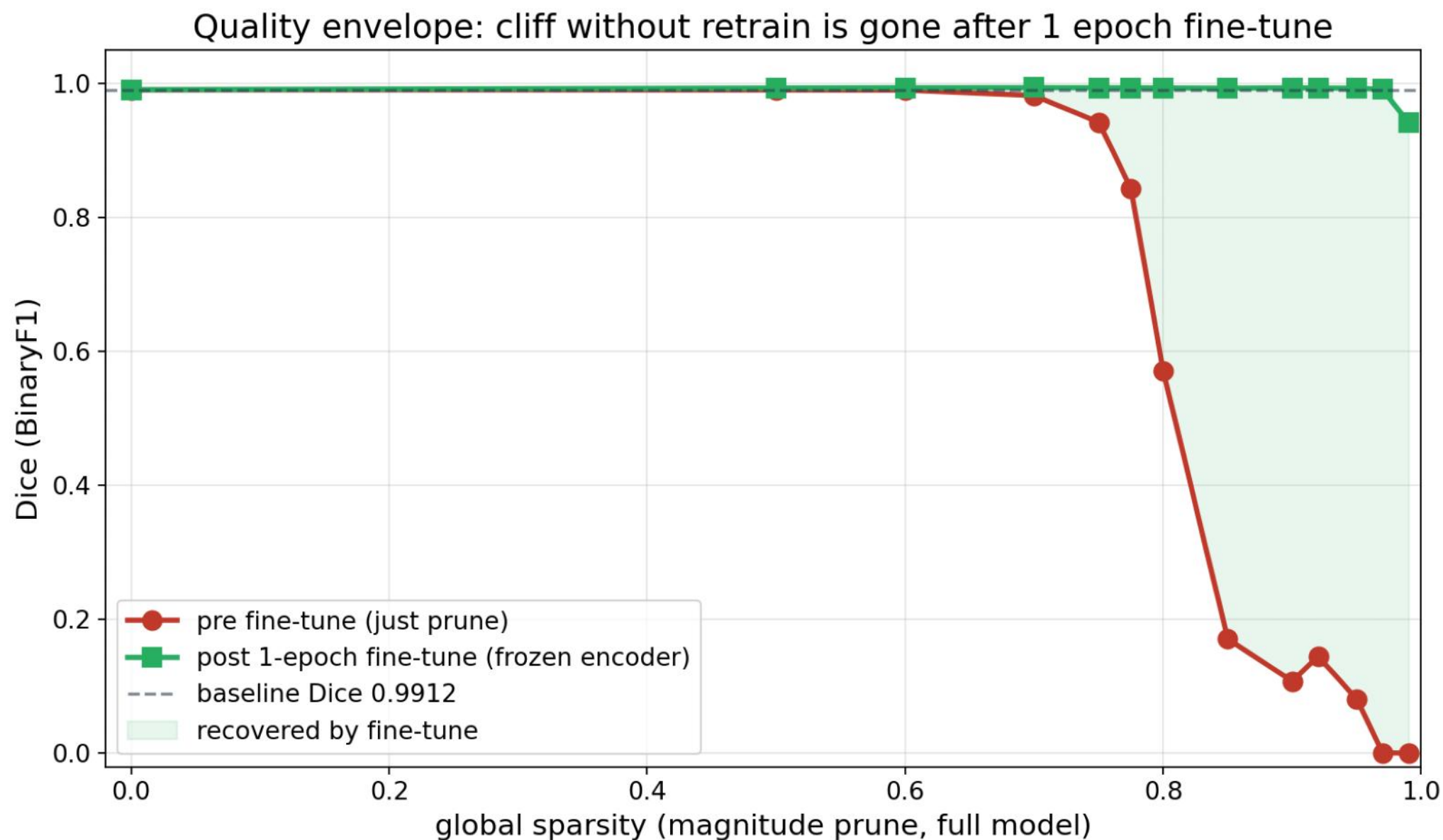
Курилов Олег

Использованные методы спарсификации

- Magnitude pruning (global L1 unstructured, `torch.nn.utils.prune`)
- 2:4 semi-structured (NVIDIA Ampere pattern, 50% sparsity)
- Structured channel pruning (per-conv L2 norm, mask-only)
- Fine-tune с frozen encoder (NVIDIA ASP recipe, 1 эпоха)
- Iterative ladder и ASP combo (поверх базовых методов)

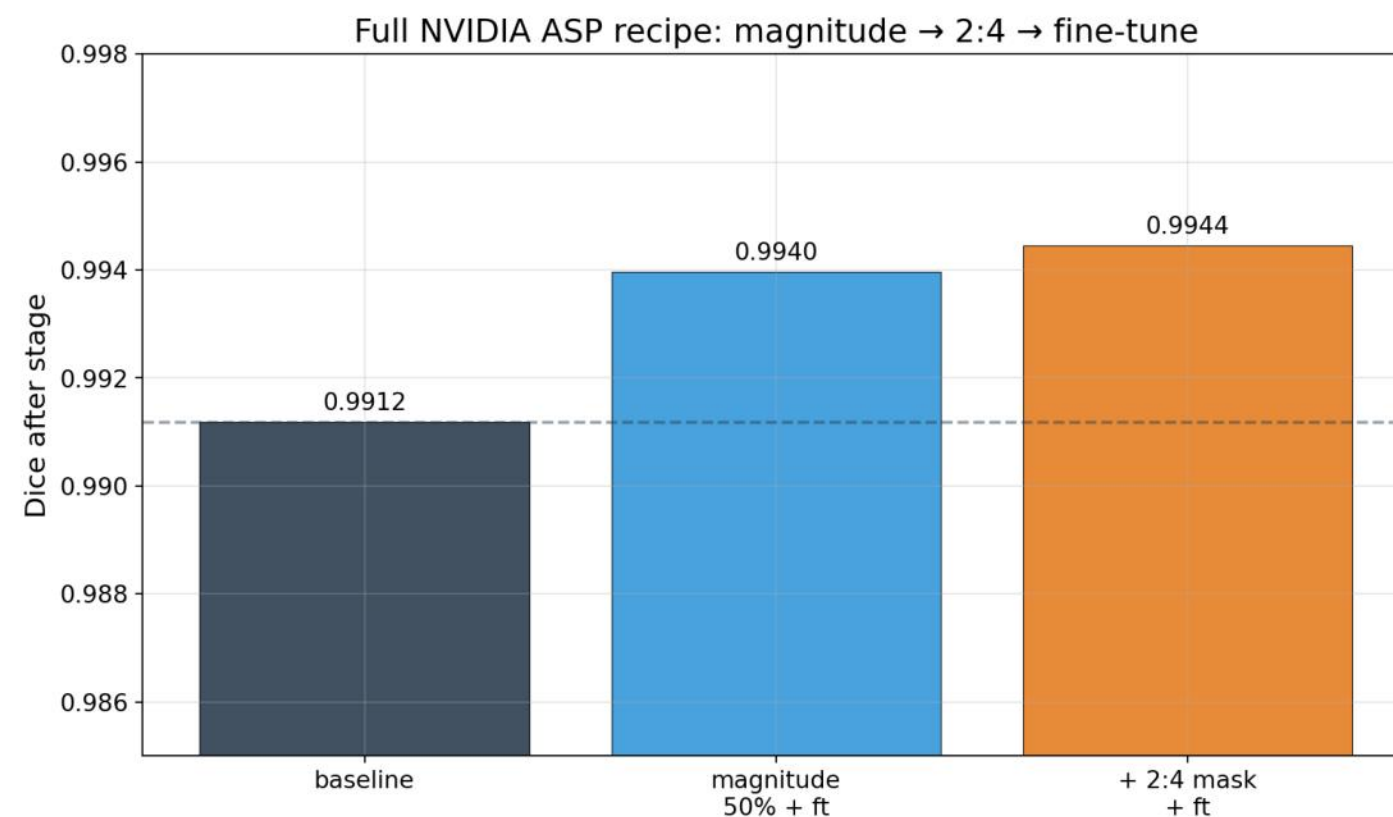
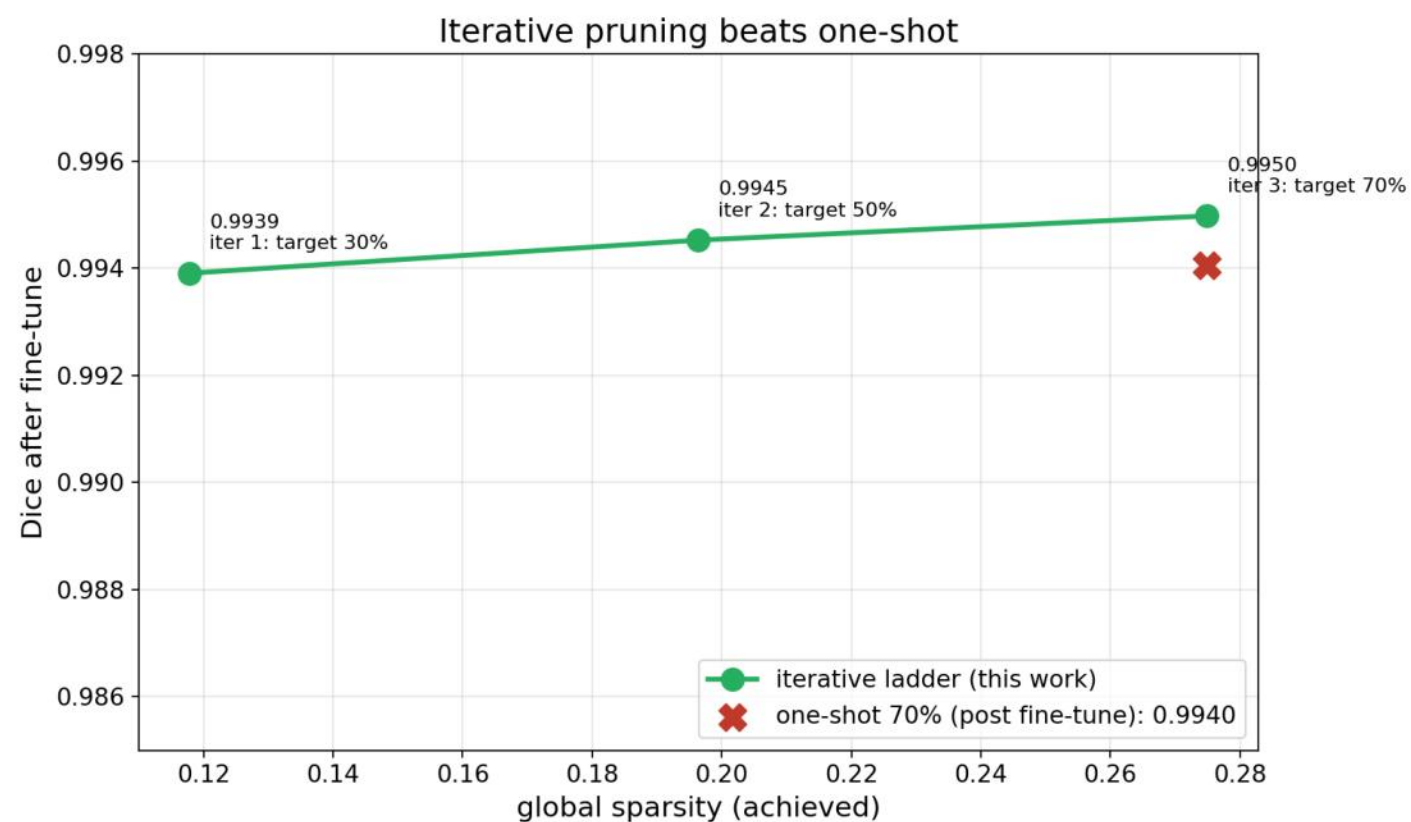
Set-up: UNet 31M, Carvana val=400, NVIDIA RTX A4000 (sm_86), fp16, bs=8

Magnitude pruning: качество vs sparsity



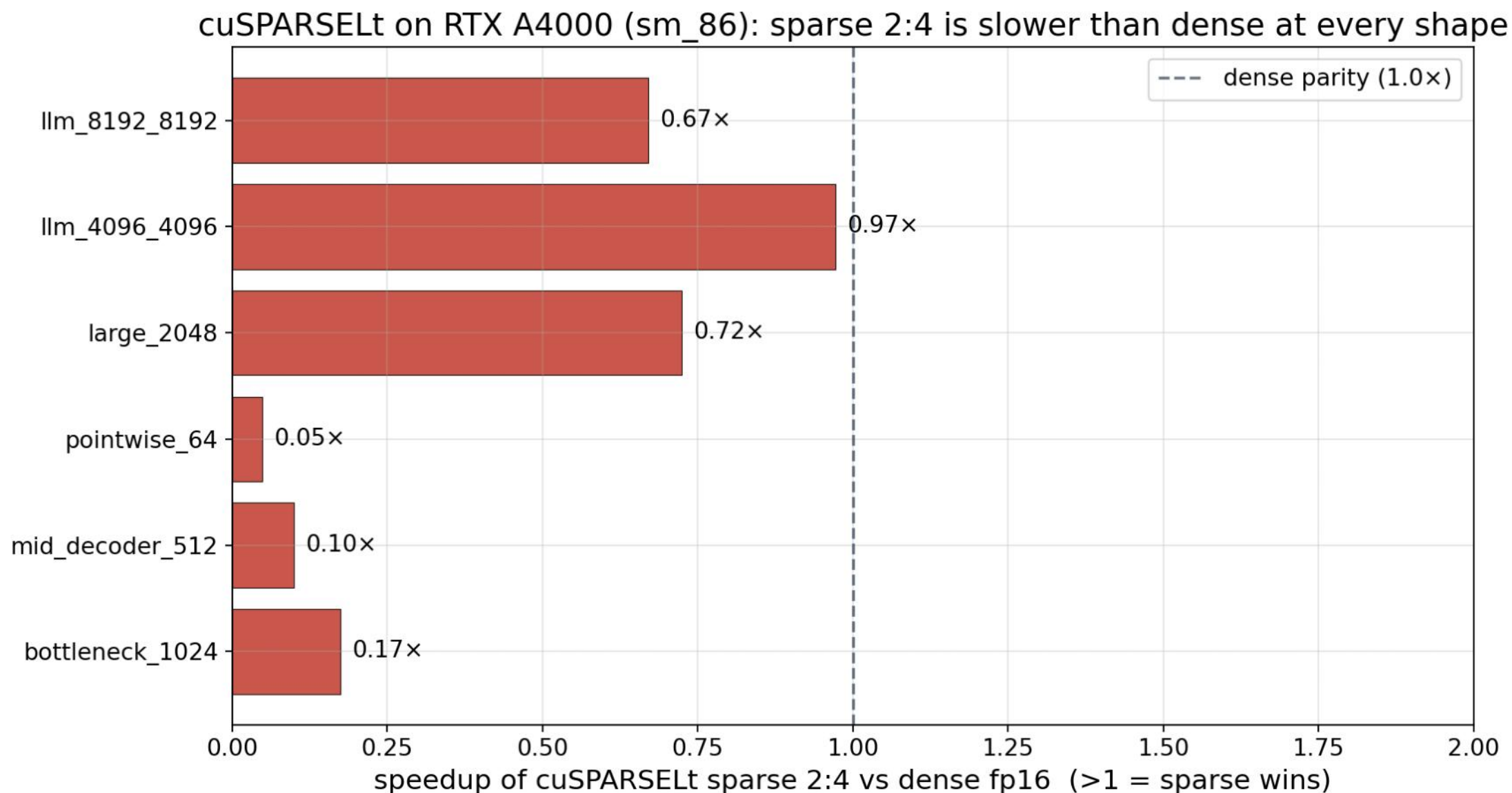
Без retrain cliff на 75-80%, после 1 эпохи fine-tune Dice держит baseline до 97%

Iterative ladder и NVIDIA ASP combo



Iterative 30→50→70% даёт +0.001 к one-shot. ASP combo: 0.991 → 0.994 при 24% sparsity

cuSPARSELt 2:4 — hardware path на A4000



На consumer Ampere (sm_86) sparse 2:4 везде медленнее dense, hardware path только на A100 (sm_80)

PyTorch Profiler и память

Self CUDA total, 4 батча bs=8:

конфиг	CUDA total	aten::conv2d
baseline_fp16	1.085 s	~95%
magnitude@0.5	1.078 s	~95%
2:4 (50%)	1.076 s	~95%

- Разница <1% между всеми тремя
- ~95% времени в aten::conv2d
- cudnn dense kernels игнорируют нули
- Для speedup нужны sparse kernels
- Peak GPU memory: 6112 MB у всех

Без sparse-aware kernels нет ни ускорения, ни экономии памяти

Итоги спарсификации

- Качество: 70% magnitude бесплатно, 97% после retrain $\geq$ baseline
- Скорость на dense fp16: ускорения нет
- cuSPARSELt 2:4 на A4000: 1.5–7x медленнее dense
- Peak GPU memory: одинаковый у всех методов (6112 MB)
- Сэкономили только размер чекпоинта (если хранить как sparse)
- Дальше: physical channel removal, перенос на A100

Спарсификация без специальных kernels уменьшает только размер чекпоинта, не runtime